correlations between them. This is depicted in Figure 19. The strong self correlation on the diagonal is of course expected, and we see an expected pattern in the values close to that diagonal. We did not, however, expect to see such a strong and regular pattern in the upper and lower diagonal regions of this matrix, even for columns that represent inputs that are very far removed from each other.

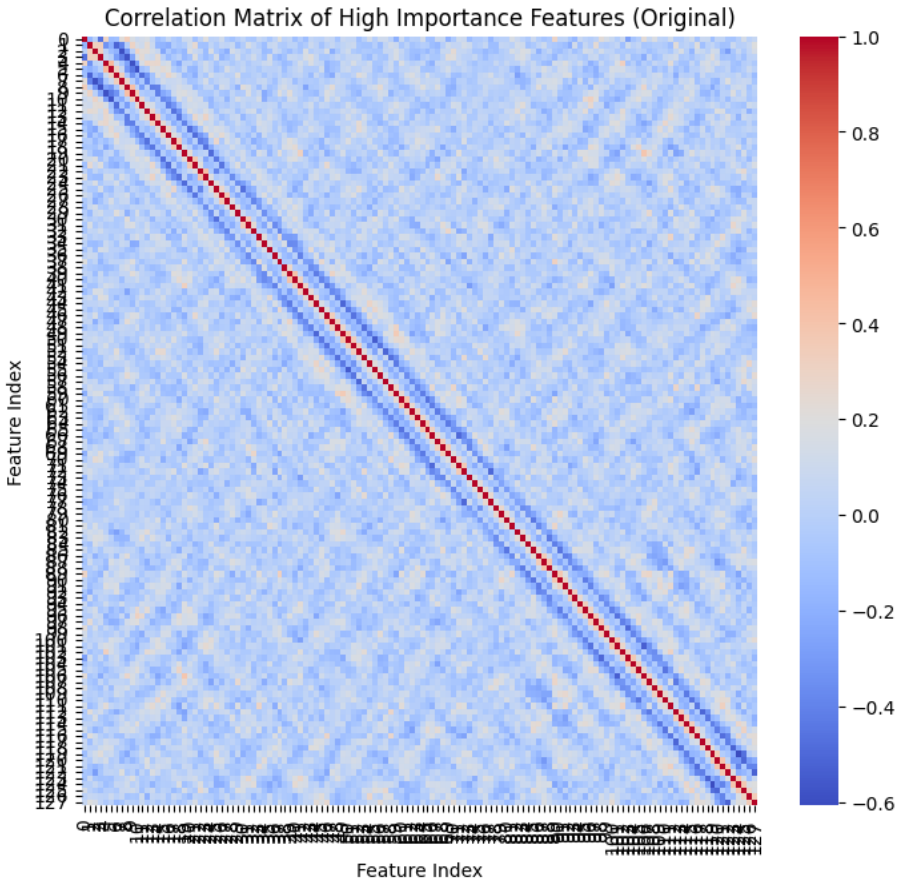

Figure 19: Correlation Matrix for columns of layer 1 weight matrix.

Our best guess for the cause of this pattern is that there is a strong preference for a new set of four consecutive ones to appear on a different neuron after the current one has finished any existing pattern. This would cause regions of alternating sequences of "more likely to code" and "less likely to code", each of length four, which is consistent with the pattern. This preference presumably is fairly strong for it to not perceptibly fade throughout the entire matrix.

6 Modeling Computation and Combinatorial Disentanglement

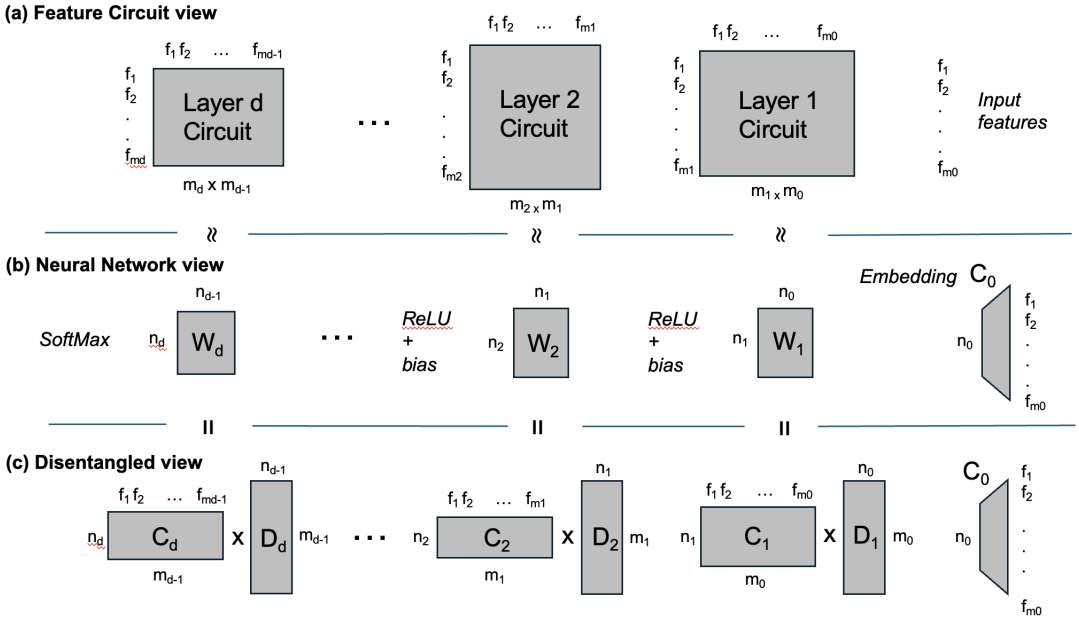

Figure 20: The feature circuit coding-decoding view of computation. Computation flows from right to left. The middle sequence (b) is the actual computation through the neural network. The top sequence (a) is the abstract computation mapping features to features in each layer being represented by the neural network. The lower sequence (c) is the feature coding view that bridges the two by thinking of the the weight matrix as implementing a decoding of the prior layer's features to serve as monosemantic inputs to a coding-and-computation of the features of the new one. $W_i = C_i D_i$ is a superposed $n_i \times n_{i-1}$ representation of the $m_i \times m_{i-1}$ i-th feature circuit layer.

Thus far we have illustrated the combinatorial view of a two-layer network. Real-world models—even plain multilayer perceptrons (MLPs)—are usually richer: they begin with an embedding layer and then stack several nonlinear layers. This section introduces a conceptual model for understanding MLP computation based on underlying "feature circuits." We then propose a novel perspective viewing MLP layers as implicitly performing a decode-compute-encode cycle, aiming to disentangle the network and reveal this circuit. Building on this, we outline a potential strategy, termed *cascading feature disentanglement*, for systematically uncovering the feature circuit layer by layer. We present initial steps demonstrating how the first layer of computation can be revealed via disentanglement from an input embedding and also propose a general approach for deeper layers.

6.1 The Feature Circuit View of Neural Computation

Our modeling follows the intuitive ideas in [1, 12, 37]. We aim to capture the fundamental computational structure implemented by an MLP. We model the computation at each layer i (for $i = 1, \ldots, d$) as producing a set of abstract output features $F_i = \{f_{i,1}, f_{i,2}, \ldots, f_{i,m_i}\}$. Each feature $f_{i,j} \in F_i$ is computed by applying a specific function to a subset of the features $F_{i-1} = \{f_{i-1,1}, \ldots, f_{i-1,m_{i-1}}\}$ from the preceding layer (where F_0 represents the initial input features, m_0 in number). The complete *feature circuit* of the network is the sequence of these computations across all layers, $F_1, F_2, \ldots, F_d$.

Figure 20(a) provides a visualization of such a multi-layer feature circuit. It takes m_0 input features and computes features layer by layer, where the computation of a feature $f_{i,j}$ depends only on a subset of features from F_{i-1}. For instance, in our earlier simple 2-layer network example (a fully connected layer followed by a single neuron computing a DNF of 2-ANDs), the feature circuit might resemble Figure 21. Note that in that figure, the rectangular portion of the matrix depicts the dependencies (using filled in rectangles), and the formulas to the left of the rectangle depict the actual functions being computed.

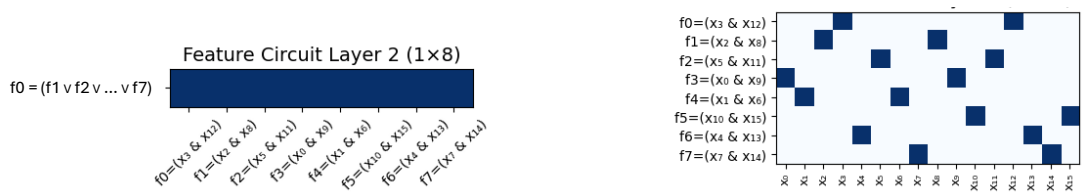

Figure 21: Two layer feature circuit (as an example of the top row of Figure 20) of the DNF $(x_3 \wedge x_{12}) \vee (x_2 \wedge x_8) \vee (x_5 \wedge x_{11}) \vee (x_0 \wedge x_9) \vee (x_1 \wedge x_6) \vee (x_{10} \wedge x_{15}) \vee (x_4 \wedge x_{13}) \vee (x_7 \wedge x_{14})$. Layer 1 defines the computation of the pairwise AND features and Layer 2 is the single OR feature over the results of the outputs from Layer 1. Together they define the abstract computation that the network in Figure 22 computes.

This abstract feature circuit finds a concrete implementation in a standard MLP, depicted in Figure 20(b). The m_0 input features are first mapped by an embedding matrix C_0 (size $n_0 \times m_0$) to an n_0-dimensional vector. Subsequent layers $i = 1, \ldots, d$ consist of a weight matrix W_i (size $n_i \times n_{i-1}$), a bias vector b_i, and a non-linearity (e.g., ReLU), transforming the n_{i-1} neuron activations from layer $i-1$ into n_i neuron activations for layer i. (While we use ReLU for illustration, other operations like batch normalization could be used instead). A final readout function (e.g., softmax) produces the network's output. Through training (e.g., via gradient descent), this MLP learns to approximate the underlying feature circuit computation.

We will say that a layer i of a neural network *computes in superposition* if $n_i < m_i$, that is, there are fewer neurons than output features in the circuit being computed by this layer. The network as a whole will be said to compute in superposition if its layers compute in superposition. Such networks are sometimes called *polysemantic* because each neuron participates in the computation of more than one output feature, as opposed to monosemantic ones, in which each neuron corresponds to a single output feature.

As [1, 37] show, a neural network can use a polysemantic representation with many fewer neurons than features to compute across multi-level neural networks. Part of our observations in this work suggest that polysemanticity can arise in training even when the network does not need to be